

Robotaksi Behavior Tree (BT) Kapsamlı İmplementasyon Raporu

Özür dilerim, önceki dokümanda teknik detayları ve sözdizimini (syntax) yüzeysel geçmişim. Bu doküman, Teknofest 2026 Robotaksi projesindeki BT mimarisinin en alt seviye C++ detaylarını, sınıf yapılarını ve istatistikleri içerir.

1. İSTATİSTİKLER: KAÇ NODE VAR, KAÇI YAPILDI?

Projemizin XML haritasında (`segment_bt.xml`) tanımlı toplam **43 adet** özel node bulunmaktadır.

- Tamamlanan (Gerçek Kod) Node Sayısı: 16**
- Stub (Henüz Bağlantısı Yapılmayan) Node Sayısı: 27**
 - Harita Bağımlı Stubs: 5
 - Sensör Bağımlı Stubs: 12
 - Nav2/Hareket Bağımlı Stubs: 10

Bu 16 node, dışarıdan başka hiçbir sensör veya harita algoritmasına ihtiyaç duymayan saf mantık, araç kontrolü ve görev kaydı node'larıdır.

2. MİMARİ YAPI VE SYNTAX TEMELLERİ

BT.CPP kütüphanesinde her node bir C++ sınıfıdır. Bir node oluşturmak için iki temel şey gerekir:

- HPP (Başlık) Dosyasında Sınıf Tanımı:** Doğru base class'tan (`ConditionNode` , `SyncActionNode` , `StatefulActionNode`) kalıtım alınmalı ve `providedPorts` statik metoduyla portlar tanımlanmalıdır.
- CPP Dosyasında `tick()` İmplementasyonu:** Node'un asıl işi yaptığı yerdir. `SUCCESS` , `FAILURE` veya `RUNNING` (sadece Stateful için) dönmelidir.

A. Portlar ve Blackboard Mimarisi

Behavior Tree'de node'lar kendi aralarında veri paylaşmak için "Blackboard" adı verilen ortak bir hafıza kullanır.

- `getInput<T>("port_adi", degisken)` : XML'den veya blackboard'dan değer okur.
- `setOutput("port_adi", deger)` : Blackboard'a değer yazar.

XML Syntax Örneği:

```
<!-- {degisken} süslü parantez, Blackboard'daki bir anahtarı işaret eder -->
<SetMaxSpeed speed="{current_max_speed}" />
```

3. TAMAMLANAN İMPLEMENTASYONLARIN DETAYLI ANALİZİ

Aşağıda, yaptığımız 16 node'un sınıf yapıları, C++ syntax'ı ve çalışma mantığı verilmiştir.

3.1. Segment Döngüsü Node'ları (`segment_loop_nodes`)

1. HasMoreSegments (ConditionNode)

- **Mimari:** `BT::ConditionNode` 'dan türetilmiştir. Saniyenin binde biri hızında kontrol yapıp biter, `RUNNING` dönemez.
- **C++ Syntax (Başlık):**

```
class HasMoreSegments : public BT::ConditionNode {
public:
    HasMoreSegments(const std::string& name, const BT::NodeConfiguration&
config) : BT::ConditionNode(name, config) {}
    static BT::PortsList providedPorts() {
        return {
            BT::InputPort<int>("seg_index"), BT::InputPort<int>("route_size")
        };
    }
    BT::NodeStatus tick() override;
};
```

- **C++ Syntax (Implementasyon):**

```
BT::NodeStatus HasMoreSegments::tick() {
    int seg_index = 0, route_size = 0;
    getInput("seg_index", seg_index);
    getInput("route_size", route_size);
    // Mantık: indeks boyuttan küçükse hala gidilecek segment var demektir
    return (seg_index < route_size) ? BT::NodeStatus::SUCCESS :
BT::NodeStatus::FAILURE;
}
```

2. AdvanceSegment (SyncActionNode)

- **Mimari:** `BT::SyncActionNode` 'dan türetilmiştir. Senkron çalışır, anında biter.
- **Port Özelliği:** `BidirectionalPort` kullanır. Yani aynı portu hem okur hem yazar.
- **Implementasyon Mantığı:** `seg_index` 'i okur, değerini 1 artırır ve `setOutput` ile Blackboard'a geri yazar.

3. ClearHandledFlags (SyncActionNode)

- **Mimari Özelliği:** Bu node port KULLANMAZ. Doğrudan Blackboard pointer'ına erişir.
- **C++ Syntax:**

```
BT::NodeStatus ClearHandledFlags::tick() {
    // config().blackboard doğrudan tree'nin ana hafızasına erişim verir
    auto bb = config().blackboard;
    bb->set("handled_stop_sign", false);
    bb->set("handled_traffic_light", false);
    return BT::NodeStatus::SUCCESS;
}
```

3.2. Araç Kontrol Node'ları (`vehicle_control_nodes`)

Bu node'lar ROS ile konuşan ilk node'larımızdır.

4. SetMaxSpeed (SyncActionNode)

- **Mimari:** Blackboard'a kaydetme + ROS publisher birleşimidir.
- **ROS Entegrasyon Syntax'ı:**

```
BT::NodeStatus SetMaxSpeed::tick() {
    double speed = 0.0;
    getInput("speed", speed);

    // 1. Blackboard'a yaz (Diğer BT node'ları okusun diye)
    config().blackboard->set("current_max_speed", speed);

    // 2. ROS Publisher oluştur ve Topic'e bas
    auto ros = getRosNode(config()); // Ortak paylaşılan node ptr
    auto pub = ros->create_publisher<std_msgs::msg::Float64>
        ("/vehicle/max_speed", 10);

    std_msgs::msg::Float64 msg;
    msg.data = speed;
    pub->publish(msg);

    return BT::NodeStatus::SUCCESS;
}
```

5. Dwell (StatefulActionNode)

- **Mimari Özelliği:** Bekleme yaptığı için `tick()` değil, `onStart()` , `onRunning()` ve `onHalted()` fonksiyonlarını kullanır.
- **C++ Syntax ve Mantiği:**

```
// İlk çalışmada çağrılır
BT::NodeStatus Dwell::onStart() {
    start_time_ = std::chrono::steady_clock::now(); // Süreyi başlat
    return BT::NodeStatus::RUNNING; // RUNNING dönerek "bitmedim, beni tekrar
    çağır" der
}

// Ağaç her ticklendiğinde (eğer RUNNING dönmüşse) bu çağrılır
BT::NodeStatus Dwell::onRunning() {
    auto elapsed = std::chrono::steady_clock::now() - start_time_;
    double sec = std::chrono::duration<double>(elapsed).count();

    if (sec >= wait_duration_sec_) return BT::NodeStatus::SUCCESS; // Süre
    doldu
    return BT::NodeStatus::RUNNING; // Henüz dolmadı
}
```

3.3. Trafik Mantık Node'ları (`traffic_logic_nodes`)

6. StopAndProceed (SyncActionNode)

- **Mimari Özelliği (Latch Mantiğı):** BT saniyede 10-20 kere tüm ağacı baştan aşağı dolaşır (tick). Eğer bir DUR tabelası gördüğümüzde durursak, bir saniye sonraki tick'te tabelayı hala görüyor olacağımız için tekrar dururuz (Sonsuz döngü). Bunu engellemek için "latch" (kalıcılık) kullanırız.
- **C++ Syntax:**

```
BT::NodeStatus StopAndProceed::tick() {
    auto bb = config().blackboard;
    bool handled = false;
    bb->get("handled_stop_sign", handled);

    // Zaten işlendiyse SUCCESS dön ve geç (tekrar durma komutu verme)
    if (handled) return BT::NodeStatus::SUCCESS;

    // İşlenmediyse bayrağı kaldır ve dur
    bb->set("handled_stop_sign", true);
    // (Burada aracı durdurma mantığı çalışır)
    return BT::NodeStatus::SUCCESS;
}
```

4. KALAN (STUB) NODE'LAR: NASIL İMPLEMENTE EDİLECEKLER?

Şu anda `stub_nodes.cpp` dosyasında bulunan 27 adet node, boş (dummy) değerler dönmektedir. Bunları implemente etmek için başka ekiplerin (Harita, Sensör) çıktıları gereklidir.

4.1. Harita Bağımlı Node'lar (Örn: `LoadMission`)

- **Ne Gerekli?** SegmentGraph C++ kütüphanesi ve `segment_map.yaml` dosyası.
- **Mimari:** SyncActionNode olacak. `tick()` içinde GeoJSON'u parse edecek, `SegmentGraph::planRoute()` fonksiyonunu çağırarak. Dönen yolu bir `std::vector<std::string>` olarak Blackboard'daki `route` portuna `setOutput` yapacak.

4.2. Sensör Bağımlı Node'lar (Örn: `PedestrianAhead`)

- **Ne Gerekli?** Algılama ekibinin yayınlayacağı ROS topic'leri (örn: `/perception/pedestrian`).
- **Mimari Kritik Nokta:** Bu node'lar `ConditionNode` 'dur. **BT'nin içinde `spin_some()` YAPILAMAZ!** Çünkü bu BT'yi kilitler.
- **Nasıl Implemente Edilmeli?**
 1. `main.cpp` içinde bir asenkron ROS thread'i çalıştırılmalı.
 2. Bu thread, `/perception/pedestrian` topic'ini sürekli dinleyip gelen veriyi Blackboard'a `is_pedestrian_detected` anahtarıyla yazmalı.
 3. `PedestrianAhead` node'u, sadece `getInput("is_pedestrian_detected")` yaparak değeri okuyup anında `SUCCESS/FAILURE` dönmelidir.

4.3. Nav2 Bağımlı Node'lar (Örn: `FollowLaneSegment`)

- **Ne Gerekli?** Nav2 `NavigateToPose` veya yerel kontrolcü Action Server'ı.
- **Mimari:** Kesinlikle `StatefulActionNode` olmalıdır.
- **Nasıl Implemente Edilmeli?**

1. `onStart()` içinde ROS Action Client üzerinden hedefe (`goal`) istek (`send_goal`) atılmalı ve `RUNNING` dönülmeli.
2. `onRunning()` içinde Action Server'dan gelen durum sorgulanmalı. Eğer hedefe varıldıysa `SUCCESS` dönmeli.
3. `onHalted()` içinde (eğer yaya çıkarsa ağaç bu node'u durdurur) Action Client'a `cancel_goal` gönderilmeli.

Bu rehber, projedeki C++ sınıflarının yapısını, BT mimarisinin kilit noktalarını ve kalan 27 node'un implementasyon haritasını eksiksiz şekilde içermektedir.